Accessing & Visualising the Knowledge Graph

Edit New page

Jump to bottom

Abdul Haseeb edited this page last week · 1 revision

Accessing & Visualising the Knowledge Graph

How to open a given user's medical knowledge graph in Neo4j and explore it visually, plus a set of ready-to-run example queries.

1. How the graph is organised

Each uploaded document is ingested by the worker pipeline (Run_Extraction_Pipeline.md) and written into a single Neo4j instance via Graphiti.

Two things you need to know before querying:

- **Every node is namespaced by `group_id`, which equals the user's Firebase UID.** One shared database holds every user's graph; the `group_id` is the only thing separating them. **Always filter by `group_id`** or you will read across patients.
- **Node labels follow a fixed medical schema.** Entities carry a generic `:Entity` label plus a type label. The full set:

  `Patient`, `Diagnosis`, `Medication`, `LabTest`, `Procedure`, `Provider`, `PathologyResult`, `TumorMarker`, `TreatmentPlan`, `ImagingResult`, `Symptom`, `Allergy`, `VitalSigns`, `Referral`, `Appointment`.

  Each document also produces one `:Episodic` node (the source record the entities were extracted from).

2. Get access to Neo4j Browser

The graph is served from a self-hosted Neo4j instance. It exposes:

- **Neo4j Browser** (the visual UI) at `http://<neo4j-host>:7474/browser/`
- **Bolt** (driver connections) at `bolt://<neo4j-host>:7687`

> **Credentials are not stored in this repository.** This repo is public - the host, username, and password are pinned in the Internal-Student-Group channel on slack.

Open the Browser URL, and connect with the Bolt URL + credentials when prompted.

3. Find a user's `group_id` (Firebase UID)

You visualise one user at a time, and every query is scoped to their UID.

1. Open the Firebase console → **Authentication** → **Users** (`https://console.firebase.google.com/project/amos26/authentication/users`).
2. Search for the user by email.
3. Copy the **User UID** column value — that is the `group_id` .

Throughout the examples below, replace `USER_UID` with this value.

4. Visualise a graph in Neo4j Browser

1. Paste a query (see below) into the command bar at the top and press **Run**.
2. When a query returns nodes and relationships, Browser renders them as an interactive graph. Use the circular **graph** toggle on the result panel if it opens in table view.
3. **Double-click any node** to expand its neighbours, or click a node to see its properties in the side panel.
4. Drag nodes to lay the graph out; scroll to zoom.

Queries that `RETURN` nodes/relationships (`n, r, m`) draw a graph. Queries that `RETURN` scalar columns (names, counts) show a table — useful for inspecting values.

5. Example queries

Replace `USER_UID` with the target user's Firebase UID. To scope to a single document, also replace `EPISODE_NAME` (see §6 for how to get it).

5.1 Full patient graph

Every entity and relationship for one user — the main "visualise my graph" view.

```
MATCH (n:Entity)-[r]-(m:Entity)
WHERE n.group_id = 'USER_UID'
RETURN n, r, m
```

5.2 Everything around the patient

The `Patient` node and everything directly connected to it.

```
MATCH (p:Entity {name: 'Patient-1', group_id: 'USER_UID'})-[r]-(n:Entity)
RETURN p, r, n
```

5.3 Entities from a single document

What one specific document contributed to the graph.

```
MATCH (ep:Episodic {name: 'EPISODE_NAME', group_id: 'USER_UID'})-[r]-(n:E
RETURN ep.name AS document, n.name AS entity, labels(n) AS entity_type
ORDER BY ep.name
```

5.4 Entity-type breakdown

How many of each medical entity type this user has (table view).

```
MATCH (n:Entity)
WHERE n.group_id = 'USER_UID'
UNWIND labels(n) AS label
WITH label WHERE label <> 'Entity'
RETURN label AS entity_type, count(*) AS count
ORDER BY count DESC
```

5.5 All documents on the timeline

Every ingested document for the user, in chronological order.

```
MATCH (e:Episodic)
WHERE e.group_id = 'USER_UID'
RETURN e.name AS document, e.valid_at AS document_date
ORDER BY e.valid_at
```

5.6 One diagnosis across documents (entity merging)

Shows how the same clinical concept is linked from multiple documents — the value of the graph over isolated reports. Replace the entity name as needed.

```
MATCH (d:Entity {name: 'C61 Prostatakarzinom', group_id: 'USER_UID'})-[r]h
RETURN d, r, other
```

> **Never paste a real patient UID or entity name into this public repository.** The examples use placeholders on purpose.

6. Shortcut: get a document's exact queries from the API

The backend already generates and stores the two most common queries for every document. `GET /documents/{document_id}` returns:

- `graph_query` — the user's full patient graph (query §5.1, pre-filled with the real UID).
- `entities_query` — the entities for that specific document (query §5.3, pre-filled with the real `EPISODE_NAME`).

So instead of assembling a query by hand, call the API for a document and copy the `entities_query` / `graph_query` value straight into Neo4j Browser. The API base is in api-config.ts; auth is a Firebase ID token (see the Document API integration guide).

Team Processes

- Publishing a release
- Issues, peer reviews and pull requests

Clone this wiki locally

```
https://github.com/amosproj/amos2026ss03-ailixir-intelligence.wiki.git
```